DATA 37711: Machine Learning & AI Part I

Homework 2

Problem 1: Conditional mean and uncertainty

(a) Link function. Write the network output as $g(x) = (g_1(x), g_2(x))$. A suitable link is

$$f_\mu(x) = g_1(x), \qquad f_\sigma(x) = \text{softplus}(g_2(x)) + \varepsilon, \quad \text{softplus}(t) = \log(1 + e^t),$$

with $\varepsilon > 0$. The mean can take any real value, while the standard deviation is strictly positive. An exponential link for the standard deviation is also valid, but softplus is more numerically stable for large outputs.

(b) Training loss. Under $Y \mid X = x \sim N(f_\mu(x), f_\sigma(x)^2)$, minimize the average negative log likelihood

$$L = \frac{1}{n} \sum_{i=1}^{n} \left[\frac{1}{2} \log(2\pi) + \log f_\sigma(x_i) + \frac{(y_i - f_\mu(x_i))^2}{2 f_\sigma(x_i)^2} \right].$$

The residual term rewards accurate means relative to their predicted uncertainty, and $\log f_\sigma$ prevents reducing loss simply by making all standard deviations large.

(c) Taxi model and baselines. The dataset contains 523,554 trips and seven already-standardized features. I use all seven supplied features and explicitly select `trip_duration` as the target. A fixed random permutation with seed 42 gives 366,487 training, 78,533 validation, and 78,534 test observations, approximately 70/15/15. No observations or extreme targets are removed.

The PyTorch network is $7 \to 64 \to 64 \to 2$, with ReLU hidden activations. The target is standardized using only the training mean $m = 10.41246$ and standard deviation $s = 7.74750$. If the network outputs (a, b), its predictions in minutes are

$$\widehat{\mu}(x) = m + s a(x), \qquad \widehat{\sigma}(x) = s\big[\,\text{softplus}(b(x)) + 0.01\big].$$

I optimize Gaussian NLL using Adam, learning rate 10^{-3}, weight decay 10^{-4}, and batches of 1,024. Training permits 100 epochs with patience 12 on validation NLL. Epoch 45 is selected and restored; training stops at epoch 57. All reported density losses include $\frac{1}{2}\log(2\pi)$ and use duration in minutes.

The regression baselines are (i) a constant Gaussian with training mean and standard deviation, and (ii) a linear mean fit by least squares, with a constant standard deviation estimated from training residuals. For the binary event, I also use the training empirical frequency as a probability baseline.

Model	Val. NLL	Test NLL	Test RMSE (min)	Test MAE (min)
Constant Gaussian	3.461440	3.476824	7.8285	5.7056
Linear Gaussian	3.434907	3.447380	7.6016	5.5879
Gaussian MLP	**2.665424**	**2.659013**	**4.0452**	**2.5045**

The MLP improves both mean prediction and Gaussian NLL over these baselines. Its nominal 90% intervals, $\widehat{\mu} \pm 1.64485\widehat{\sigma}$, cover 94.55% of test observations. They are conservative overall, so the fitted Gaussian uncertainty is not perfectly calibrated.

(d) Probability of a trip shorter than 45 minutes. Standardizing the fitted normal distribution gives

$$\widehat{p}(x) = P(Y < 45 \mid X = x) = \Phi\left(\frac{45 - \widehat{\mu}(x)}{\widehat{\sigma}(x)}\right),$$

where Φ is the standard normal CDF. The classifier predicts the event when $\widehat{p}(x) \geq 0.5$.

(e) Held-out evaluation. For $z_i = \mathbf{1}\{y_i < 45\}$, I report two proper scoring rules:

$$\text{Brier} = \frac{1}{n}\sum_i (\widehat{p}_i - z_i)^2, \qquad \text{LogLoss} = -\frac{1}{n}\sum_i \left[z_i \log \widehat{p}_i + (1 - z_i)\log(1 - \widehat{p}_i)\right].$$

For numerical stability, probabilities are clipped to $[10^{-7}, 1 - 10^{-7}]$ before scoring. Accuracy uses the fixed threshold 0.5.

Model	Test Brier	Test log loss	Test accuracy
Constant Gaussian	0.004495	0.055854	99.5505%
Linear Gaussian	0.004494	0.046257	99.5505%
Empirical event frequency	0.004475	0.028780	99.5505%
Gaussian MLP	**0.002998**	**0.017267**	**99.6447%**

The empirical probability is 0.995599, estimated from training labels only. In the test data, 99.5505% of trips already last less than 45 minutes, so accuracy alone is not informative: the MLP improves it by only 0.0942 percentage points. It detects 112 of the 353 trips lasting at least 45 minutes. Its lower Brier score and log loss provide stronger evidence that the input-dependent probabilities improve over a constant event probability.

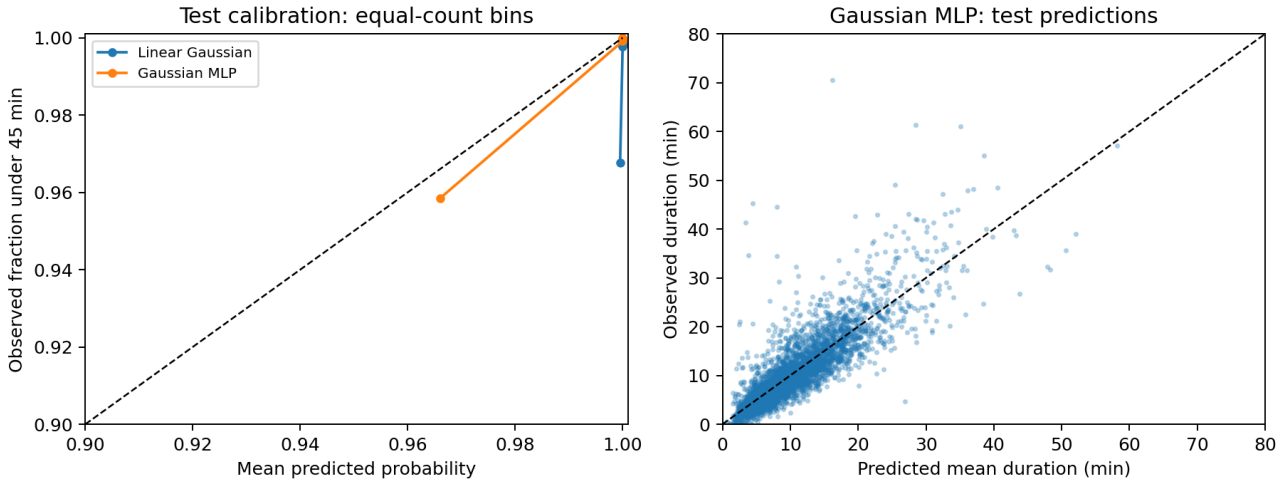


Figure 1: Test calibration in ten equal-count probability bins and a random sample of test mean predictions. Calibration is shown near one because the event is very common.

The calibration plot also shows residual probability errors. This random row split measures performance on the supplied distribution and does not evaluate generalization to future time periods.

Problem 2: Encoding categorical variables

(a) Cardinality and preparation. The retail data contain 397,924 transactions, with no missing values in the supplied columns.

Feature	Unique values	Unique in training	Singleton categories
StockCode	3,665	3,595	167
CustomerID	4,339	4,311	72

One-hot encoding would have 8,004 columns and only two nonzero entries per row. A dense float32 matrix for the full dataset would require about 11.86 GiB, so it is inefficient. Sparse one-hot encoding is feasible. Target encoding gives two numerical features, while embeddings provide compact learned category representations.

I use a random 70/15/15 split with seed 42: 278,546 training, 59,689 validation, and 59,689 test observations. Category vocabularies, encodings, and preprocessing statistics are learned from training data. Only validation loss controls early stopping.

Quantity is strongly right-skewed: the median is 6, mean 13.02, 99th percentile 120, and maximum 80,995. I train all three models with squared error on $z = \log(1 + Y)$:

$$L = \frac{1}{n} \sum_i \left(z_i - \widehat{z}_i\right)^2.$$

This reduces the influence of enormous orders. I predict $\widehat{Y} = \max\{1, \exp(\widehat{z}) - 1\}$ and report log-space MSE, its square root (RMSLE), and raw-scale MAE/RMSE. The inverse-transformed prediction targets log-space accuracy; it is not generally the arithmetic conditional mean of Quantity.

(b) Target encoding. For each feature separately, I compute a smoothed arithmetic mean of raw Quantity:

$$\mathrm{TE}(c) = \frac{\sum_{i:X_i=c} Y_i + 20\overline{Y}}{n_c + 20},$$

where $\overline{Y}$ and n_c are computed from the relevant training portion. Each training row receives a five-fold out-of-fold encoding, excluding every target in its own fold. Validation and test use maps fitted on the entire training split. Unseen categories receive the global training mean. This prevents a row's target from leaking into its own input.

I apply $\log(1+\mathrm{TE})$ and standardize the two inputs using the out-of-fold training statistics. A $2 \to 64 \to 32 \to 1$ ReLU network predicts z. Its selected checkpoint is epoch 13, giving test log MSE 0.402013 and RMSLE 0.634045.

(c) Embedding model. Each feature has its own 16-dimensional trainable embedding. Their concatenation feeds a $32 \to 64 \to 32 \to 1$ ReLU network. Categories are assigned indices using training data only; unseen categories use a fixed zero vector. The test set has 51 rows with unseen StockCodes and 16 with unseen CustomerIDs, possibly overlapping. The selected checkpoint is epoch 23. Test log MSE is 0.268072 and RMSLE is 0.517756, improving log MSE by about 33.3% over target encoding.

(d) Combined model. I concatenate the two target-encoded inputs and the 32 embedding coordinates, then fit a $34 \to 64 \to 32 \to 1$ ReLU network. It uses the same out-of-fold training encodings as part (b). All three networks use Adam, learning rate 0.003, weight decay 10^{-4}, batch size 2,048, at most 40 epochs, and patience 6. Each restores its best validation checkpoint; the hybrid selects epoch 18.

Model	Val. log MSE	Test log MSE	Test RMSLE	Test MAE	Test RMSE
Constant	1.010325	0.996641	0.998319	10.2061	41.3628
Target encoding	0.400340	0.402013	0.634045	6.9660	32.3021
Embeddings	0.267290	**0.268072**	**0.517756**	**5.7792**	30.4097
Combined	**0.264604**	0.268360	0.518035	5.8948	**29.6528**

The constant baseline predicts the training mean of z, then applies the same inverse transform. All reported test values use the untouched test split. The combined model has the lowest validation error, but embeddings alone are slightly better on test log MSE. The test difference is only 0.000288, so there is no clear gain from combining them on the primary metric. The combined model has the lowest raw RMSE, showing that the ranking depends on the metric.

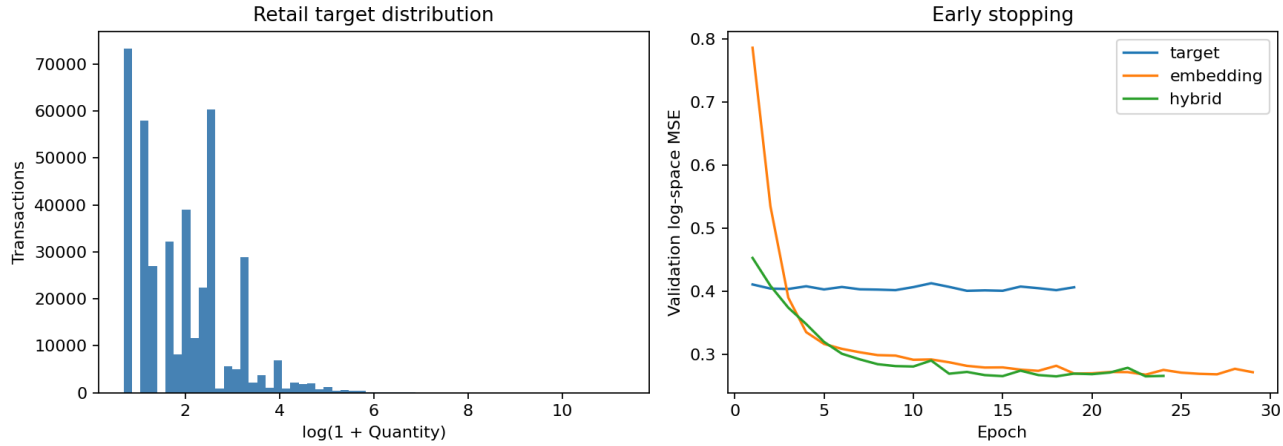


Figure 2: The transformed retail target and validation learning curves. The selected checkpoints precede the end of each curve.

(e) Interpretation. Target encoding collapses each category into one average. Two products with the same average order size become indistinguishable under this encoding, even if they interact differently with individual customers. An embedding gives each category several coordinates, learned jointly with the prediction network. The network can use these coordinates to represent useful category properties and nonlinear customer–product interactions.

The much lower test error of the embedding model suggests that category-specific information beyond two marginal means is useful here. The combined model supplies both summary statistics and learned representations, but some of this information is redundant. Its small validation improvement does not translate into a clear improvement in test RMSLE.

This random transaction split primarily evaluates customers and products already seen during training. It does not establish performance for entirely new customers/products or future time periods. The results come from one split and seed, so the tiny difference between embeddings and the combined model should not be treated as statistically established.

Problem 3: Random Kitchen Sinks and a one-hidden-layer MLP

(a) Training data. The supplied splits contain 600 training observations, 200 validation observations, and 200 unlabeled test inputs. The colored scatter plot on the next page shows a smooth response: a broad low region near $(x_1, x_2) = (-1.5, 0)$, increasing values toward $|x_2| = 3$, and a curved variation along x_1.

(b) Random Kitchen Sinks. I draw $h = D/2$ Gaussian projections $w_j \sim N(0, I/\sigma^2)$ and phases $b_j \sim U[0, 2\pi]$. The features are

$$\phi(x) = \frac{1}{\sqrt{h}}\big(\sin(w_1^T x + b_1), \cos(w_1^T x + b_1), \ldots, \sin(w_h^T x + b_h), \cos(w_h^T x + b_h)\big)^T.$$

The factor $1/\sqrt{h}$ keeps the overall feature scale comparable across widths. I fit an unpenalized intercept and ridge coefficients by minimizing

$$\sum_{i \in \text{train}} \big(y_i - a - \beta^T \phi(x_i)\big)^2 + \lambda \|\beta\|_2^2.$$

Only D varies; $\sigma = 1$ and $\lambda = 10^{-3}$ remain fixed. The validation minimum occurs at $D = 50$, or $h = 25$. Increasing D further lowers training error but slightly increases validation error, consistent with overfitting.

(c) One-hidden-layer MLP. I use a $2 \to m \to 1$ network with a tanh hidden activation and a linear output. Inputs and the target are standardized using training-set statistics; predictions are returned to the original target scale. I vary only the hidden width m. All models use Adam with learning rate 0.01, batch size 600, 4,000 epochs, and weight decay 10^{-5}. Initialization uses seed 42. The lowest validation MSE occurs at $m = 256$, although $m = 16$ is almost tied. Width therefore does not have a simple monotone relationship with validation performance in this run.

RKS sweep

D	Train MSE	Val. MSE
10	0.778920	0.710045
20	0.058264	0.056286
50	0.032399	**0.037138**
100	0.031681	0.037500
200	0.030800	0.038400
400	0.030247	0.038517
800	0.029581	0.040825
1200	0.029477	0.041498

MLP sweep

m	Train MSE	Val. MSE
8	0.033222	0.036675
16	0.032935	0.036101
32	0.031560	0.037585
64	0.031941	0.037584
128	0.032663	0.036467
256	0.032829	**0.036007**

(d) Final fits and predictions. After selection, I refit both models on the training split using their selected hyperparameters and the same seed. Validation labels remain outside fitting. RKS has validation MSE 0.037138 and MLP has 0.036007, a roughly 3.0% lower MSE for the MLP. Its hidden features adapt to the response, whereas the RKS features are fixed random draws. Both fit this smooth function well; the small difference on one validation split does not establish a general advantage.

The output files are `rks_test_predictions.csv` and `mlp_test_predictions.csv`. Each contains the header `prediction` followed by 200 predictions in the original test-row order. Test MSE cannot be computed because those labels are withheld.

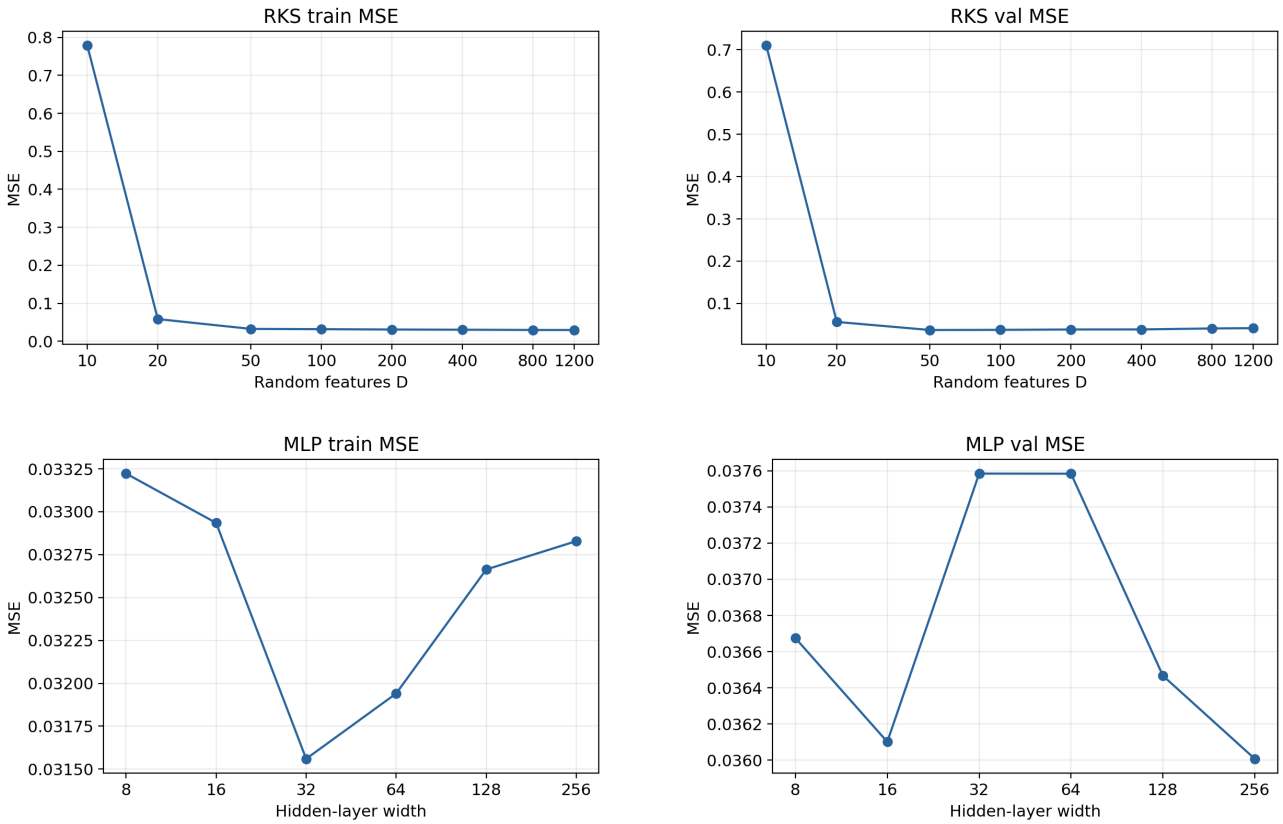


Figure 3: Training and validation MSE across the two hyperparameter sweeps.

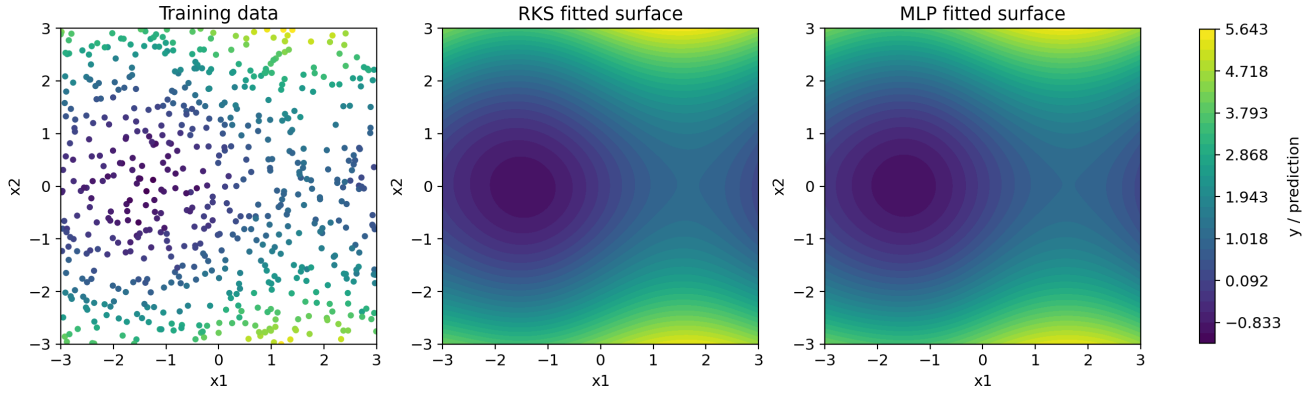


Figure 4: Training observations and the selected fitted functions over $[-3, 3]^2$, using a common color scale.

Both surfaces recover the broad trough, high values near the top and bottom boundaries, and the curved pattern across x_1. Neither follows the pointwise noise. The RKS surface is slightly more rounded near the left boundary, while the MLP has learned a similar shape with adaptive hidden features. These similarities agree with their close validation errors.